FATEC CAMPINAS

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

KEZIA CALIXTO

Banco de dados do Prazo Claro

Concepção modelagem e operação da infraestrutura

Relatório técnico complementar ao Trabalho de Graduação II
Orientação do Prof. Eduardo Felipe Fonseca de Freitas

Versão para revisão da autora e do orientador
Campinas SP
4 de novembro de 2026

Roteiro de leitura

1 Contexto e necessidades de informação
2 Modelo conceitual e recorte implementado
3 Engenharia de dados e decisões estruturais
4 Modelo lógico e físico
5 Dicionário de dados
6 Integridade índices e consultas
7 Segurança do acesso ao banco
8 Funcionamento do histórico
9 Funcionamento transacional dos lembretes
10 Infraestrutura e operação hospedada
11 Evolução do schema e entrega
12 Evidências e limites da verificação
13 Conclusão
Apêndice A Consultas de inspeção sem dados pessoais
Apêndice B Perguntas para explicar o modelo
Apêndice C Catálogo de funções
Referências e rastreabilidade

As entradas acima são vínculos internos. O relatório reúne modelo conceitual e físico, dicionário completo, segurança, fluxos e infraestrutura. A versão descrita conserva o aceite humano do lembrete de 24/10 e não implica nova inspeção de produção em novembro.

Apresentação do relatório

Relatório técnico de apoio ao Trabalho de Graduação II de Kezia Calixto, do curso de Análise e Desenvolvimento de Sistemas da FATEC Campinas, sob orientação do Prof. Eduardo Felipe Fonseca de Freitas. Edição de 4 de novembro de 2026. Base documental do repositório em 49c256d e base funcional publicada identificada em 9632fd2.

O banco do Prazo Claro sustenta a preservação das simulações, o isolamento do histórico e a execução dos lembretes. A solução combina PostgreSQL, Supabase Auth, Data API e políticas de segurança por linha. O motor processual recebe versões de regras e calendários e calcula sem consultar o banco. Essa separação permite testar a contagem de modo determinístico e conservar resultados antigos mesmo quando a aplicação evolui.

As quatro migrations descrevem sete tabelas do projeto: cinco no schema public e duas no schema reminder_private. O levantamento estático identificou 65 colunas, sete índices declarados explicitamente, sete políticas RLS e 22 funções em sua definição final. Esses números excluem tabelas, índices e funções gerenciados pela plataforma, como auth.users, objetos do Vault e estruturas das extensões. O inventário acompanha este relatório para permitir conferência por arquivo e linha. [P01]

Este documento explica a decisão de persistência desde a concepção até a operação hospedada. O modelo físico deriva dos arquivos SQL; a infraestrutura e os resultados operacionais derivam dos registros identificados do projeto. Não foi executada consulta ao banco de produção nesta edição. A confirmação humana de recebimento e abertura de memória do lembrete, registrada em 24/10/2026, está incorporada; não se apresenta a infraestrutura como novamente auditada em novembro. [P08]

1 Contexto e necessidades de informação

1.1 Problema que orientou a persistência

Uma simulação processual depende de entradas, marcos, regra, calendário e condições de aplicação. Guardar apenas a data final eliminaria informações necessárias para explicar por que um dia entrou na contagem e outro foi excluído. O problema de dados é, portanto, conservar o contexto do cálculo e permitir sua consulta posterior sob a identidade do proprietário.

O sistema também precisa distinguir a vida da simulação da vida do aviso. Alterar o estado de um lembrete não pode modificar a memória original. Por isso, cálculo e lembrete possuem registros próprios. O agendador, por sua vez, precisa de dados operacionais de curta retenção, sem se transformar em um segundo histórico de conteúdo processual. [P02]

Os requisitos de persistência abrangem salvar e consultar memórias, separar contas, preservar versões, permitir exclusão e oferecer programação opcional de e-mail. Não se exige, nesta entrega, cadastro de partes, documentos judiciais, números de processo ou colaboração entre escritórios. Essa delimitação reduz dados pessoais e evita anunciar estruturas de equipe ainda sem políticas e testes próprios.

1.2 Evolução da concepção

O master prompt original propunha um domínio mais amplo, com organizações, membros, tribunais, unidades, fontes, versões, eventos e agrupamento de cálculos por processo. O projeto implementou primeiro o histórico individual. O ADR 0003 registra o adiamento de organizações e memberships até existir uma especificação de papéis, pertencimento e migração dos registros já salvos. [P03]

O ADR 0002 definiu acesso pela Data API com supabase-js e clientes SSR, adiando o ORM Drizzle inicialmente previsto. Essa escolha concentrou o acesso corrente em requisições que transportam a identidade da sessão e chegam ao PostgreSQL com políticas RLS. A ausência de ORM não eliminou SQL, chaves, índices ou migrations; esses elementos permaneceram explícitos e versionados. [P03]

Em 20/10, a migration inicial criou histórico e estrutura de catálogos. Em 23/10, três migrations acrescentaram fila, separação dos modos local e Brevo e agendador hospedado. Cada mudança ampliou a operação sem exigir substituir a memória dos cálculos anteriores. A confirmação de recebimento em 24/10 encerrou um aceite operacional específico dos lembretes. [P01] [P08]

1.3 Requisitos que moldaram o banco

Requisitos de dados e decisões de implementação

Necessidade	Decisão aplicada	Efeito verificável
Histórico individual	owner_id associado a auth.users	Sessão acessa somente os próprios registros
Explicar a estimativa	input e result_snapshot em JSONB	Entrada, versões e trilha acompanham o resultado
Conservar o passado	Ausência de UPDATE para usuários	A sessão não edita o snapshot persistido
Regras e calendários	Chaves estáveis e versão única	O modelo diferencia identidade e revisão
Aviso opcional	Tabela deadline_reminders e RPCs	Programação não altera a memória original
Concorrência	Índice único parcial e claim transacional	Uma geração ativa por cálculo e reserva exclusiva
Operação hospedada	Configuração e invocações privadas	Agendamento e diagnóstico ficam fora da API pública

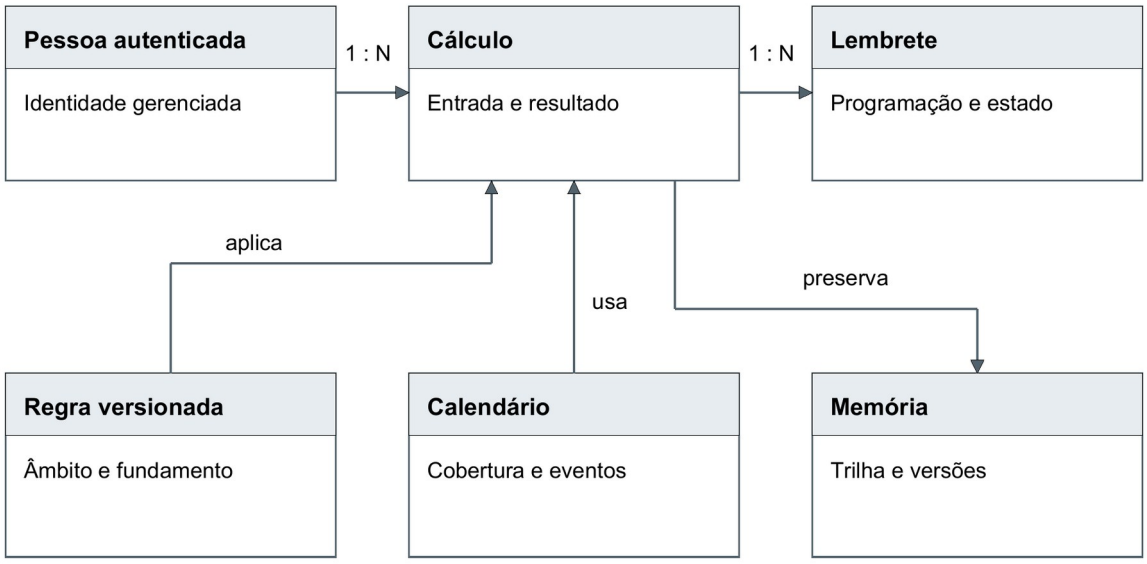
Fonte: requisitos, ADRs, migrations e rotas do projeto. [P01] [P02] [P03] [P04]

2 Modelo conceitual e recorte implementado

2.1 Entidades do domínio

No nível conceitual, a pessoa autenticada possui cálculos, e um cálculo pode originar gerações de lembrete. A simulação usa uma regra e um calendário identificados por versão e preserva a memória produzida. As fontes fundamentam as regras e os eventos considerados. A figura 1 expressa essas relações de significado, sem afirmar que cada entidade conceitual possui uma tabela autônoma.

Figura 1 Modelo conceitual do recorte de persistência



Relações conceituais; os componentes de memória não exigem tabelas separadas.

Fonte Elaboração a partir da DDL e da implementação versionadas do Prazo Claro.

Na implementação, usuário corresponde à identidade gerenciada por Supabase Auth. Evento processual, marcos e trilha diária são componentes dos documentos JSONB do cálculo. Eventos do calendário e referências normativas também podem integrar snapshots. Não há tabelas independentes ProceduralEvent, CalculationTrace ou CalendarEvent nas quatro migrations. [P01] [P02]

Organização, membership, processo completo, perfil administrativo de revisão e preferência global de notificações permanecem extensões planejadas. Elas não aparecem como tabelas existentes nos diagramas físicos. Se forem acrescentadas, precisarão de regras de pertencimento e migração que conservem a propriedade atual dos registros. [P03]

2.2 Relação entre catálogo de código e catálogo persistente

As tabelas legal_sources, legal_rule_versions e calendar_versions constituem a estrutura relacional para fontes e versões. Contudo, o catálogo ativo da calculadora está nos arquivos TypeScript do domínio. A aplicação seleciona regras e calendários locais versionados antes de calcular; não busca automaticamente uma linha dessas três tabelas para realizar cada simulação. [P02] [P04]

O seed versionado permanece intencionalmente vazio. Os testes criam fontes e calendários fictícios em transações descartáveis para verificar políticas. A existência das tabelas e dos estados pending, approved e rejected não comprova aprovação jurídica nem funcionamento de

um painel de administração normativa. Essa distinção é essencial para explicar a concepção completa e o produto entregue.

3 Engenharia de dados e decisões estruturais

3.1 Modelo relacional com documentos JSONB

O desenho utiliza relações para identidades, propriedade, vínculos e estados operacionais. Usa JSONB nos componentes cuja estrutura precisa conservar a memória do domínio: entrada, resultado, escopo, definição de regra, calendário e metadados. Essa composição evita criar tabelas de detalhe antes de existir uma necessidade de consulta relacional sobre cada dia da trilha. [P01]

O PostgreSQL valida a sintaxe JSONB e disponibiliza operadores para acesso aos campos. JSONB não conserva a representação textual original, a ordem das chaves ou duplicatas de chave. Portanto, a expressão memória preservada significa conservação do conteúdo semântico armazenado, não cópia byte a byte do JSON enviado nem assinatura digital. A exportação reflete o objeto preservado pela aplicação. [T01]

A validação é distribuída. CHECKs verificam o tipo de topo dos objetos e alguns limites de tamanho. Os schemas Zod na aplicação verificam a estrutura interna antes de apresentar a memória ou autorizar um lembrete. O banco não contém uma validação JSON Schema exaustiva de todas as propriedades do resultado. [P01] [P04]

3.2 Normalização e redundância deliberada

Chaves primárias identificam registros independentemente do texto visível. Fontes e versões de regra usam relações explícitas para evitar repetição de metadados normativos. Identidade e conteúdo do cálculo ficam separados da fila de envio. A configuração do agendador e seus registros de invocação também têm tabelas distintas, pois possuem ciclo de vida diferente.

Existem redundâncias intencionais. O lembrete guarda owner_id além de calculation_id para autorização e consulta direta, e copia due_date e snapshot_metadata no momento da programação. O cálculo conserva uma cópia de regras e calendários usados em seu snapshot, em vez de depender apenas de referências para registros que possam mudar. O objetivo é estabilidade histórica e operação controlada, não afirmar normalização integral de todo JSONB. [P01] [P03]

As duas FKs do lembrete, isoladamente, garantem a existência do cálculo e da conta, mas não que o proprietário informado seja o proprietário daquele cálculo. Essa igualdade é verificada pela função de agendamento, que consulta o cálculo sob owner_id da identidade autenticada antes de inserir. Não há FK composta para impor essa igualdade em qualquer escrita

administrativa. A responsabilidade por essa condição deve continuar explícita em futuras rotas privilegiadas.

3.3 Tipos datas e estados

UUID identifica entidades e tokens de reserva; TEXT armazena rótulos e estados; BOOLEAN registra decisões simples. Os estados usam TEXT com CHECK, e não tipos ENUM personalizados. DATE representa dias civis, como vencimento e cobertura. TIMESTAMPTZ representa instantes de criação, envio e processamento; o fuso America/Sao_Paulo orienta a transformação da data e hora de lembrete em um instante. [P01]

O PostgreSQL não usa uma coluna TIMESTAMPTZ para conservar o nome do fuso originalmente escolhido. Por isso, o lembrete mantém time_zone separadamente e o restringe a America/Sao_Paulo. A data do vencimento permanece DATE, evitando tratá-la como um instante UTC. A antecedência do lembrete é contada em dias corridos do calendário, sem alterar a regra processual do cálculo.

3.4 Identidade de versão e preservação

legal_rule_versions possui unicidade em rule_key e version; calendar_versions possui unicidade em calendar_key e version. Essa combinação permite novas versões sem reutilizar a mesma identidade lógica de revisão. Fontes possuem UUID próprio e estado de revisão. A vigência e a cobertura impedem intervalos finais anteriores aos iniciais. [P01]

Na memória histórica, a aplicação compara engineVersion, ruleVersion, calendarId, calendarVersion, actId, actVersion e countingEngineVersion antes de associar um recálculo compatível. O resultado original continua separado do recomputed. Versão antiga indisponível não é silenciosamente substituída por uma versão atual. A coluna status dos cálculos aceita somente provisional; resultados incompletos não são salvos pela rota de histórico. [P04]

4 Modelo lógico e físico

4.1 Schemas e fronteiras de responsabilidade

Schemas e estruturas relacionadas

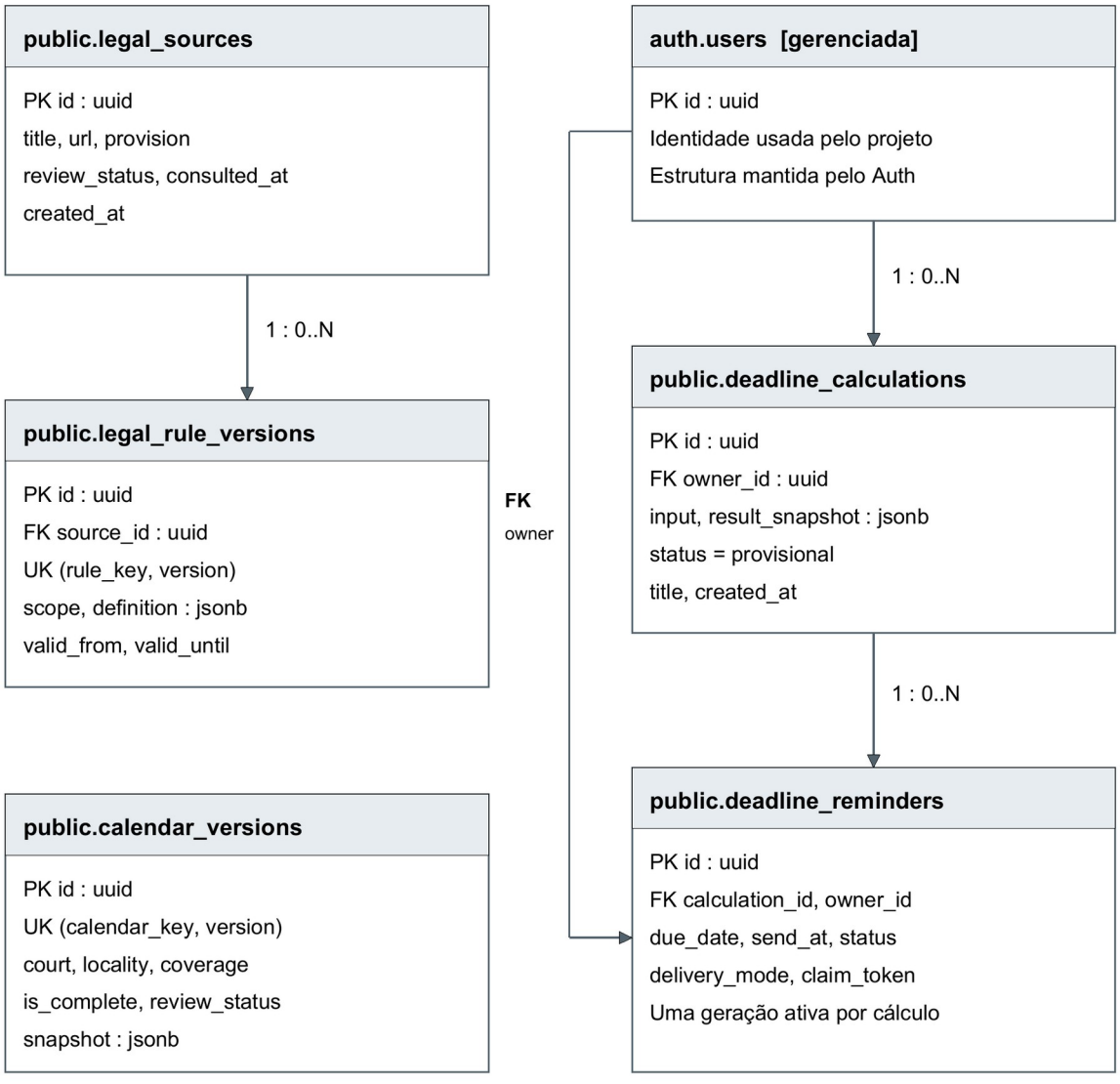
Schema	Responsabilidade	Controle do projeto
public	Cinco tabelas do domínio e wrappers RPC	Definidos pelas migrations e permissões explícitas
reminder_private	Duas tabelas e implementações internas	Fora dos schemas expostos na configuração local
auth	Identidades e sessões do Supabase Auth	Gerenciado pela plataforma; consumido sob contrato
vault	Segredo dedicado ao agendador	Armazenamento gerenciado; valor não integra o relatório
cron	Job e histórico de execução do pg_cron	Operação administrativa do agendador
net	Fila HTTP e respostas do pg_net	Objetos da extensão, sem reescrever grants da plataforma
extensions	Funções de extensões usadas na integração	Não corresponde a tabelas de negócio do projeto

Fonte: SQL versionado, config.toml e ADR 0006. [P01] [P03] [P05]

O schema public não torna automaticamente todos os dados públicos: exposição, GRANTs e políticas são controles diferentes. A configuração local expõe public e graphql_public e define auto_expose_new_tables=false. A análise não presume que cada configuração local foi sincronizada automaticamente com o dashboard hospedado.

4.2 Relacionamentos efetivamente impostos

Figura 2 Diagrama físico das tabelas públicas e da identidade gerenciada



Setas: pai para filho por FK. Campos resumidos; o dicionário lista todas as colunas.

Fonte Elaboração a partir da DDL e da implementação versionadas do Prazo Claro.

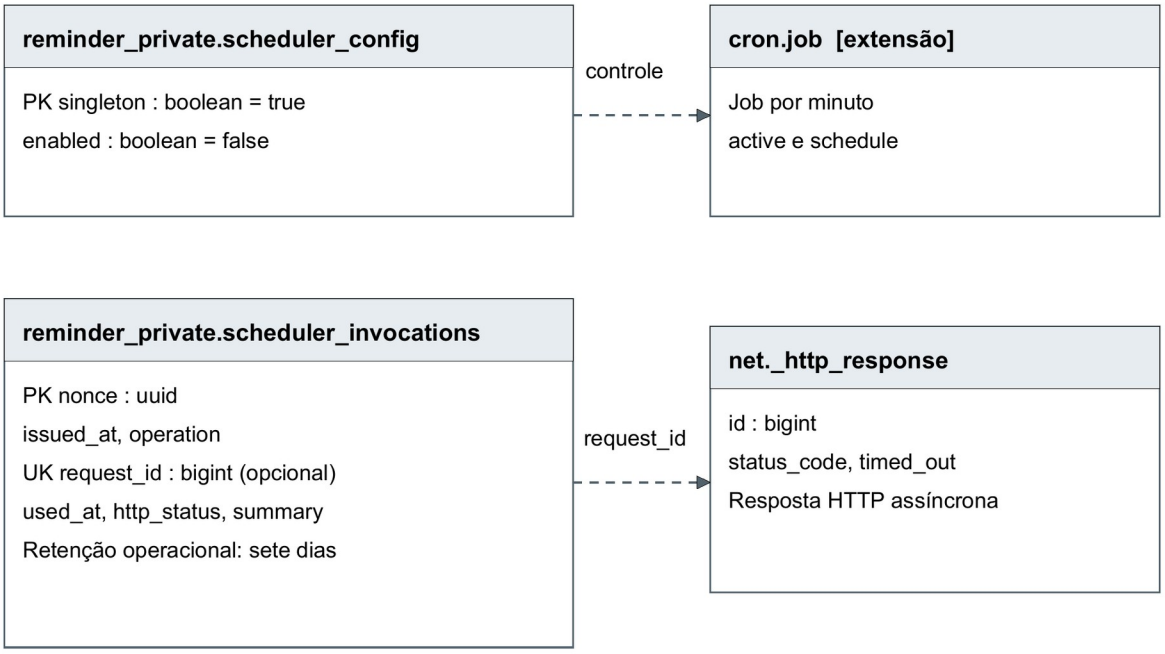
A figura 2 mostra somente quatro relacionamentos implementados por FK: legal_rule_versions.source_id para legal_sources.id; deadline_calculations.owner_id para auth.users.id; deadline_reminders.calculation_id para deadline_calculations.id; e deadline_reminders.owner_id para auth.users.id. Cada filho exige exatamente um pai e cada pai pode ter zero ou vários filhos. [P01]

A exclusão de uma fonte referenciada é restringida. A exclusão de um cálculo remove seus lembretes por cascata. A exclusão administrativa de uma conta pode remover cálculos e

lembretes ligados a ela; isso não equivale a existir tela de exclusão da conta. calendar_versions não possui FK para legal_sources, e o snapshot do cálculo não possui FK para as tabelas de versão. Essas associações aparecem no documento JSONB e no contrato de aplicação, sem garantia relacional equivalente.

4.3 Estruturas privadas do agendador

Figura 3 Modelo de dados operacionais do agendador e correlação HTTP



Fonte Elaboração a partir da DDL e da implementação versionadas do Prazo Claro.

scheduler_config usa singleton BOOLEAN como chave e CHECK(singleton). Assim, existe no máximo uma linha, cuja chave é true; enabled começa false. scheduler_invocations usa nonce UUID como chave, issued_at como timestamp numérico, operation limitada a dispatch ou probe e request_id BIGINT único quando preenchido.

request_id é usado para correlacionar a invocação com net._http_response.id durante a coleta. Não há FK para net._http_response. Também não há FK do nonce para um lembrete: uma invocação pode despachar um lote e guarda somente contagens e status operacionais mínimos. A figura usa linha tracejada para a correlação por valor, distinguindo-a de integridade referencial.

5 Dicionário de dados

O dicionário a seguir inclui todas as 65 colunas das sete tabelas próprias, extraídas das migrations. NN significa NOT NULL, PK indica chave primária e FK indica chave estrangeira. Campos opcionais admitem NULL quando não houver informação ou quando a etapa operacional ainda não tiver ocorrido. O tipo exibido conserva o SQL de origem. Auth, Vault e extensões não são reinventados como tabelas próprias. [P01]

public.legal_sources

Campo	Tipo SQL	Nulo	Finalidade e regra
id	uuid	Não	PK. UUID gerado na inserção.
title	text	Não	Rótulo de exibição com limite de tamanho.
url	text	Não	URL da fonte; prefixo HTTPS exigido.
provision	text	Não	Dispositivo ou trecho de referência normativa.
consulted_at	timestamptz	Não	Instante de consulta à fonte.
review_status	text	Não	pending, approved ou rejected.
created_at	timestamptz	Não	Instante de criação; padrão now().

public.legal_rule_versions

Campo	Tipo SQL	Nulo	Finalidade e regra
id	uuid	Não	PK. UUID gerado na inserção.
rule_key	text	Não	Identidade lógica estável da regra.
version	text	Não	Identidade da revisão no catálogo.
source_id	uuid	Não	FK para legal_sources; exclusão restrita.
scope	jsonb	Não	Objeto com âmbito de aplicação.
valid_from	date	Não	Início da vigência.
valid_until	date	Sim	Fim opcional, não anterior ao início.
definition	jsonb	Não	Objeto com definição da regra.
review_status	text	Não	pending, approved ou rejected.
created_at	timestamptz	Não	Instante de criação; padrão now().

public.calendar_versions

Campo	Tipo SQL	Nulo	Finalidade e regra
id	uuid	Não	PK. UUID gerado na inserção.
calendar_key	text	Não	Identidade lógica do calendário.
version	text	Não	Identidade da revisão no catálogo.
court	text	Não	Tribunal ao qual a versão se refere.
locality	text	Não	Localidade declarada na cobertura.
coverage_start	date	Não	Primeiro dia da cobertura.
coverage_end	date	Não	Último dia; não anterior ao início.
is_complete	boolean	Não	Completude; necessária para approved.
review_status	text	Não	pending, approved ou rejected.
snapshot	jsonb	Não	Objeto com a versão do calendário.
created_at	timestamptz	Não	Instante de criação; padrão now().

public.deadline_calculations

Campo	Tipo SQL	Nulo	Finalidade e regra
id	uuid	Não	PK. UUID gerado na inserção.
owner_id	uuid	Não	FK para auth.users; cascata na exclusão.
title	text	Não	Rótulo de exibição com limite de tamanho.
input	jsonb	Não	Objeto com entrada do cálculo; até 32 KiB em texto.
result_snapshot	jsonb	Não	Objeto de memória original; até 1 MiB em texto.
status	text	Não	Somente provisional.
created_at	timestamptz	Não	Instante de criação; padrão now().

public.deadline_reminders

Campo	Tipo SQL	Nulo	Finalidade e regra
id	uuid	Não	PK. UUID gerado na inserção.
calculation_id	uuid	Não	FK para cálculo; cascata na exclusão.
owner_id	uuid	Não	FK para auth.users; cascata na exclusão.
due_date	date	Não	Vencimento civil capturado ao agendar.
lead_days	integer	Não	Antecedência de 0, 1 ou 3 dias corridos.
send_hour	integer	Não	Hora local admitida: 9, 12 ou 18.
time_zone	text	Não	Valor fixo America/Sao_Paulo.
send_at	timestamp with time zone	Não	Instante planejado para envio.
snapshot_metadata	jsonb	Não	Cópia dos metadados originais de versão.
status	text	Não	pending processing accepted failed cancelled uncertain.
attempt_count	integer	Não	Número de reservas; entre zero e três.
next_attempt_at	timestamp with time zone	Não	Instante mínimo para nova tentativa elegível.
claim_token	uuid	Sim	Token da reserva; necessário a prepare e settle.
claimed_at	timestamp with time zone	Sim	Instante da reserva para controlar lease.
accepted_at	timestamp with time zone	Sim	Instante do aceite pelo serviço, se houver.
error_code	text	Sim	Código fixo opcional, limitado por CHECK.
created_at	timestamp with time zone	Não	Instante de criação; padrão now().
delivery_mode	text	Não	local ou brevo; legado começa local.

reminder_private.scheduler_config

Campo	Tipo SQL	Nulo	Finalidade e regra
singleton	boolean	Não	PK com CHECK true; no máximo uma linha.
enabled	boolean	Não	Habilitação operacional; começa false.

reminder_private.scheduler_invocations

Campo	Tipo SQL	Nulo	Finalidade e regra
nonce	uuid	Não	PK. Identificador de uso único da invocação.
issued_at	bigint	Não	Timestamp de emissão em segundos; positivo.
operation	text	Não	dispatch ou probe.
created_at	timestamp with time zone	Não	Instante de criação; padrão now().
used_at	timestamp with time zone	Sim	Marca do consumo único da invocação.
request_id	bigint	Sim	ID HTTP único quando preenchido; sem FK.
http_status	integer	Sim	Status HTTP coletado, quando disponível.
completed_at	timestamp with time zone	Sim	Instante de conclusão da coleta.
timed_out	boolean	Sim	Flag de timeout quando a conclusão é conhecida.
summary	jsonb	Não	Objeto de diagnóstico mínimo permitido.

6 Integridade índices e consultas

6.1 Restrições de conteúdo

As constraints rejeitam título de fonte fora de 1 a 300 caracteres, URL de fonte sem prefixo HTTPS, título de cálculo fora de 1 a 120 caracteres e intervalos de vigência ou cobertura invertidos. Objetos JSONB exigidos não podem ser substituídos por arrays ou escalares. No cálculo, input serializado em texto tem limite de 32.768 bytes e result_snapshot tem limite de 1.048.576 bytes. Esses limites de conteúdo não correspondem ao tamanho físico total de armazenamento da linha. [P01]

Um calendário aprovado exige is_complete=true. Isso é uma coerência de estados do banco; não representa conferência jurídica automática. Nos lembretes, lead_days aceita 0, 1 ou 3; send_hour aceita 9, 12 ou 18; attempt_count aceita de zero a três. time_zone tem um único valor admitido. error_code é opcional e, quando presente, deve pertencer ao conjunto declarado no SQL.

Regras adicionais aparecem nas funções: e-mail confirmado, cálculo pertencente ao solicitante, snapshot elegível e programação futura. O servidor faz validações antes de chamar a RPC e o banco repete as verificações críticas dentro da operação que grava a programação.

Há limites diferentes por fronteira: o parser de corpo HTTP usa 32 KB, e a memória gerada no servidor pode ser maior do que a entrada recebida.

6.2 Índices explícitos

Índice explícito	Definição resumida
legal_rule_versions_source_idx	public.legal_rule_versions(source_id)
deadline_calculations_owner_created_idx	public.deadline_calculations(owner_id, created_at desc)
deadline_reminders_calculation_idx	public.deadline_reminders USING btree (calculation_id)
deadline_reminders_one_active_idx	public.deadline_reminders USING btree (calculation_id) WHERE (status = ANY (ARRAY['pending'::text, 'processing'::text, 'uncertain'::text]))
deadline_reminders_owner_created_idx	public.deadline_reminders USING btree (owner_id, created_at DESC)
deadline_reminders_pending_idx	public.deadline_reminders USING btree (next_attempt_at, send_at) WHERE (status = 'pending'::text)
scheduler_invocations_created_idx	reminder_private.scheduler_invocations USING btree (created_at)

Fonte: sete comandos CREATE INDEX das quatro migrations. PKs e constraints UNIQUE também criam estruturas de suporte, mas não estão contados como índices explícitos no quadro. [P01] [T03]

O índice único parcial de deadline_reminders impede duas gerações em pending, processing ou uncertain para o mesmo calculation_id. Ele vale entre os modos local e brevo, pois delivery_mode não integra a chave. Gerações anteriores accepted, failed ou cancelled podem coexistir no histórico. Assim, a cardinalidade total cálculo para lembrete é 1:N, com uma restrição adicional de no máximo um lembrete ativo.

O índice owner_id, created_at DESC apoia acesso e ordenação do histórico. O código também ordena por id DESC para desempate, mas o índice atual não inclui esse terceiro campo. O filtro de regime acessa input->>regime, sem índice de expressão ou GIN declarado. Essas escolhas não devem ser descritas como consultas universalmente otimizadas: índices adicionais dependeriam de volume, plano de execução e medição.

6.3 Paginação e consulta da geração mais recente

A API do histórico aceita páginas de 1 a 1000 e pageSize até 50, com padrão 10. Filtros de proprietário, regime e cálculo são aplicados antes da paginação e da contagem. O fallback após exclusão que esvazia uma página refaz a contagem sob o mesmo escopo. A busca textual exibida pela interface ocorre apenas nos registros da página carregada. [P04]

A consulta de lembretes recebe de um a 50 identificadores e usa DISTINCT ON(calculation_id), com ordem por criação e ID decrescentes, para devolver a geração mais recente de cada cálculo. A variante por modo restringe também delivery_mode. A listagem não devolve claim_token ou e-mail do destinatário ao navegador.

7 Segurança do acesso ao banco

7.1 Identidade sessão e autorização

As rotas privadas usam o cliente SSR e auth.getUser() para verificar a identidade. owner_id não é escolhido por um campo arbitrário enviado pelo navegador. A aplicação filtra os registros pela conta validada e o banco repete a restrição por RLS. As políticas também recusam sessões de sign-in anônimo, mesmo que usem o papel authenticated. [P04] [P06]

GRANT determina se um papel pode executar uma operação sobre um objeto; RLS determina quais linhas essa operação pode alcançar. Uma política não substitui permissões de tabela, e uma chave pública não transforma dados privados em conteúdo autorizado. A credencial privilegiada de lembretes existe somente no servidor e não é usada pelo histórico corrente da sessão. [T02]

7.2 Matriz de permissões de aplicação

Acesso previsto pelos objetos versionados

Objeto ou operação	anon	authenticated	Serviço de lembretes
Catálogos públicos	SELECT de aprovados	SELECT de aprovados	Fora do fluxo normal de envio
Cálculos	Sem SELECT INSERT DELETE	SELECT INSERT DELETE próprios	Sem uso direto no dispatcher
UPDATE de cálculos	Sem concessão	Sem concessão	Não integra o protocolo de envio
SELECT de lembretes	Sem concessão	Apenas próprios sob RLS	Sem SELECT direto de tabela
Agendar listar cancelar	Sem EXECUTE	RPCs autorizadas e validações internas	Sem EXECUTE nas RPCs de proprietário
Claim prepare settle	Sem EXECUTE	Sem EXECUTE	RPCs exclusivas do serviço
Consumir nonce	Sem EXECUTE	Sem EXECUTE	RPC exclusiva do serviço
Ativar job e ler operação privada	Sem acesso	Sem acesso	Não concedido ao serviço pela migration

Fonte: permissões explícitas, políticas, funções e testes SQL. A matriz descreve as concessões necessárias ao protocolo; não transforma o papel administrativo postgres em um usuário comum. [P01] [P07]

Os três catálogos possuem políticas de SELECT para aprovados. A regra exige que sua fonte também esteja aprovada, e o calendário exige completude. Não existem grants de escrita normativa para anon ou authenticated. As sete tabelas próprias têm RLS habilitada; as duas tabelas privadas não possuem políticas de leitura para clientes da API, decisão de proteção e não autorização pública implícita.

7.3 Funções privilegiadas e wrappers

As funções públicas de operação são wrappers SECURITY INVOKER, por padrão, que delegam a implementações de reminder_private. As funções internas que precisam de privilégios usam SECURITY DEFINER, search_path vazio e nomes de objetos qualificados. EXECUTE público é revogado e concedido somente aos papéis necessários. As funções de agendamento e cancelamento ainda verificam a identidade dentro do corpo. [P01]

SECURITY DEFINER não elimina a necessidade de controle: seu proprietário pode exercer privilégios que a RLS de um usuário comum não oferece. Por isso, o relatório documenta tanto a autorização da função quanto suas condições internas. O administrador do banco e quem possui credencial privilegiada permanecem componentes confiáveis do desenho.

7.4 Limite de autenticidade da memória

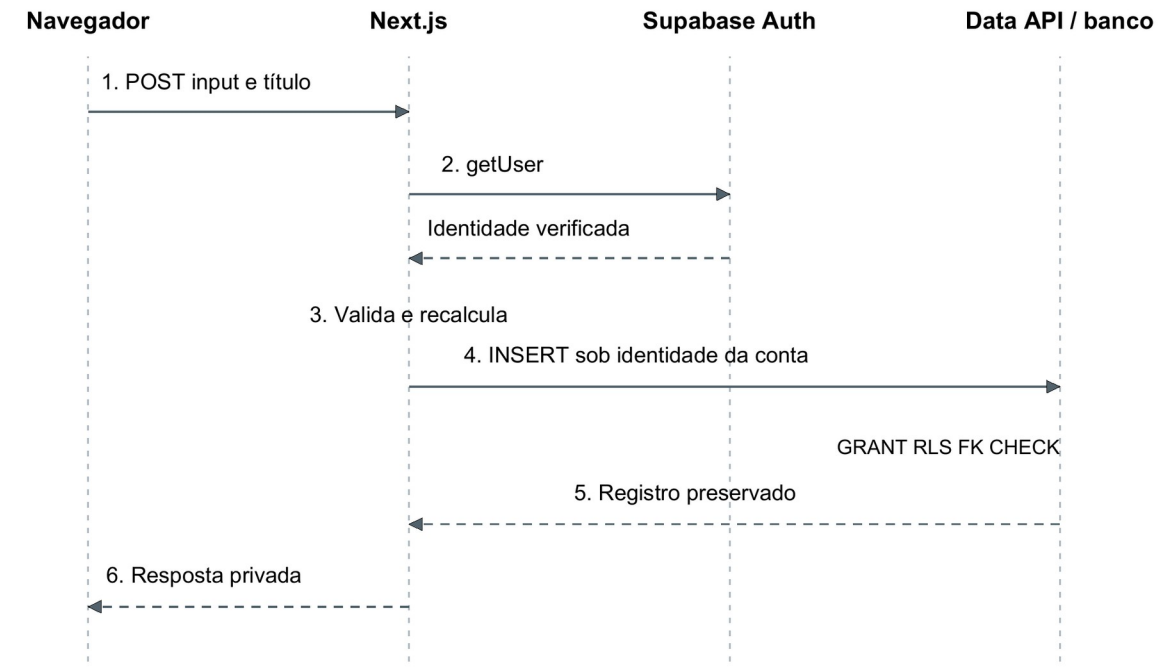
A aplicação reexecuta a entrada ao salvar, mas uma sessão autorizada ainda pode inserir seu próprio JSONB diretamente na Data API, desde que respeite propriedade, formato de topo e constraints. Por esse motivo, o histórico identifica a memória como integrity: unverified. Não existe assinatura de autenticidade, armazenamento WORM ou trilha de auditoria imutável criada apenas pelo uso de JSONB e pela ausência de UPDATE. [P06]

A segurança implementada restringe acesso entre contas e alterações pelo fluxo comum. Ela não autoriza apresentar um snapshot como prova independente de prazo processual ou como conteúdo que nem um administrador poderia alterar. A revisão jurídica e os limites do calendário continuam visíveis na memória e no e-mail.

8 Funcionamento do histórico

8.1 Gravação de uma simulação

Figura 4 Percurso de gravação e leitura de um cálculo privado



Fonte Elaboração a partir da DDL e da implementação versionadas do Prazo Claro.

No POST de /api/history, o servidor verifica origem e sessão, valida input e título e chama calculateForMvp. Se o resultado for incomplete, a rota retorna erro 422 e não insere linha. Se for provisional, a aplicação inclui owner_id da conta verificada e grava input e result_snapshot junto com o título. O PostgreSQL avalia grants, RLS, FKs e CHECKs antes de concluir o INSERT. [P04]

A gravação do histórico não consulta o relógio para decidir quais dias entram na contagem. O relógio é usado em operações de aplicação, como classificação visual do vencimento e programação do aviso. Assim, a persistência não cria dependência temporal dentro do motor determinístico.

8.2 Leitura e preservação do resultado original

No GET, a aplicação lê somente os campos necessários, valida defensivamente o snapshot e calcula uma situação comparativa: futuro, hoje, passado ou indeterminado. Essa situação depende da data atual em São Paulo e não é uma coluna permanentemente atualizada no banco. Um cálculo passado não é automaticamente prova de ato perdido ou descumprido.

Quando as versões do snapshot coincidem com as disponíveis na aplicação, um recálculo pode ser apresentado em campo separado. Sem compatibilidade, recomputed permanece nulo e a memória original continua disponível. JSON e impressão conservam os avisos e as referências pertinentes, sem converter o original em resultado aprovado. [P04]

8.3 Exclusão e efeitos da cascata

O DELETE valida UUID, origem e proprietário, além de depender da política de exclusão. Remover um cálculo remove suas linhas de lembrete por FK em cascata. Essa ação não recolhe um e-mail já aceito pelo provedor nem apaga exportações baixadas. Durante ensaio de recebimento, manter o registro até concluir a conferência evita eliminar a evidência de programação antes do envio.

O caso de teste das 09h de 24/10 ficou inconclusivo porque o responsável excluiu o cálculo sem recordar o horário. O ensaio posterior das 18h teve recebimento e acesso à memória confirmados. Esse histórico mostra por que operação, persistência e confirmação humana precisam de registros distintos. [P08]

9 Funcionamento transacional dos lembretes

9.1 Programação e captura do contexto

A função schedule valida identidade não anônima com e-mail confirmado, parâmetros admitidos e existência do cálculo sob o proprietário. O SELECT do cálculo usa FOR UPDATE. A função exige snapshot provisório com data elegível e metadata, converte a data de vencimento menos lead_days para a hora escolhida em São Paulo e recusa envio passado. A linha é inserida com estado pending e cópia dos metadados originais. [P01]

schedule_for_mode chama a programação existente e fixa delivery_mode dentro da mesma transação. A migration desse campo usa default local para não promover registros anteriores automaticamente ao envio externo. No servidor, Brevo exige consentimento explícito e configuração de produção. Essa confirmação de interface é um controle da aplicação; não existe coluna autônoma de consentimento ou auditoria imutável do opt-in na tabela. [P04]

9.2 Reserva concorrente e preparação

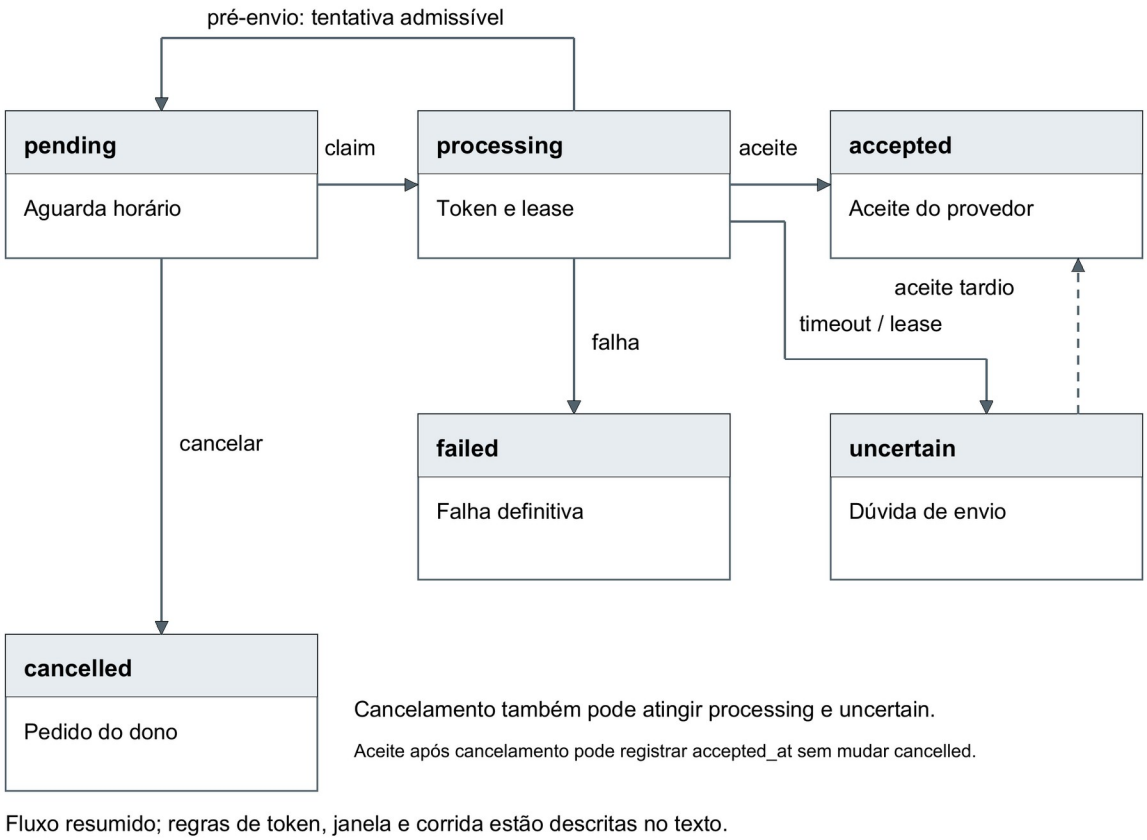
claim_for_mode usa uma CTE que seleciona candidatos vencidos para envio, com próximo horário de tentativa alcançado e menos de três tentativas. FOR UPDATE SKIP LOCKED reserva linhas sem esperar por linhas já tomadas por outro consumidor. A atualização altera status para processing, cria claim_token, registra claimed_at e incrementa attempt_count na mesma operação transacional. [P01] [T04]

A função SQL admite lotes de um a 20. O dispatcher atual solicita cinco por chamada, limite que também aparece no schema de validação da resposta. Não é correto confundir a capacidade da RPC com o tamanho do lote usado pela aplicação. Antes de reservar, leases com mais de cinco minutos passam a uncertain e pendências fora da janela de uma hora podem passar a failed.

prepare confere ID, token, status e lease. Em seguida, busca o e-mail confirmado em auth.users, excluindo destinatário indisponível, conta anônima, excluída ou com bloqueio vigente, e recupera o snapshot sob cálculo e proprietário correspondentes. O endereço não fica duplicado como coluna do lembrete e não é fornecido pelo navegador para escolher outra pessoa.

9.3 Envio liquidação e resultado incerto

Figura 5 Estados e transições principais da fila de lembretes



Fonte Elaboração a partir da DDL e da implementação versionadas do Prazo Claro.

O dispatcher verifica o provedor e envia HTTP fora da transação do banco. Depois chama settle com o token e o resultado. accepted registra aceitação e accepted_at. Uma falha anterior ao início do POST pode retornar a pending com intervalo de cinco minutos, respeitando três tentativas e a janela. Rejeição definitiva pode produzir failed, e dúvida após início do envio produz uncertain. [P01] [P04]

Transação SQL e aceitação HTTP não são uma operação atômica única. Um timeout pode ocorrer depois de o provedor aceitar a mensagem. O protocolo não promete exactly once de ponta a ponta: o estado incerto evita uma repetição automática sem evidência suficiente. A aplicação envia uma chave de idempotência baseada no ID ao Brevo, mas a segurança da fila não depende de afirmar uma garantia universal do provedor. [P03]

Cancelamento muda apenas estados ativos do proprietário. Se um envio já iniciado for aceito após o cancelamento, settle pode conservar status cancelled e registrar accepted_at. Essa combinação não deve ser apagada da análise: expressa uma corrida real entre cancelamento e processamento externo. O sistema não promete recolher mensagens em trânsito.

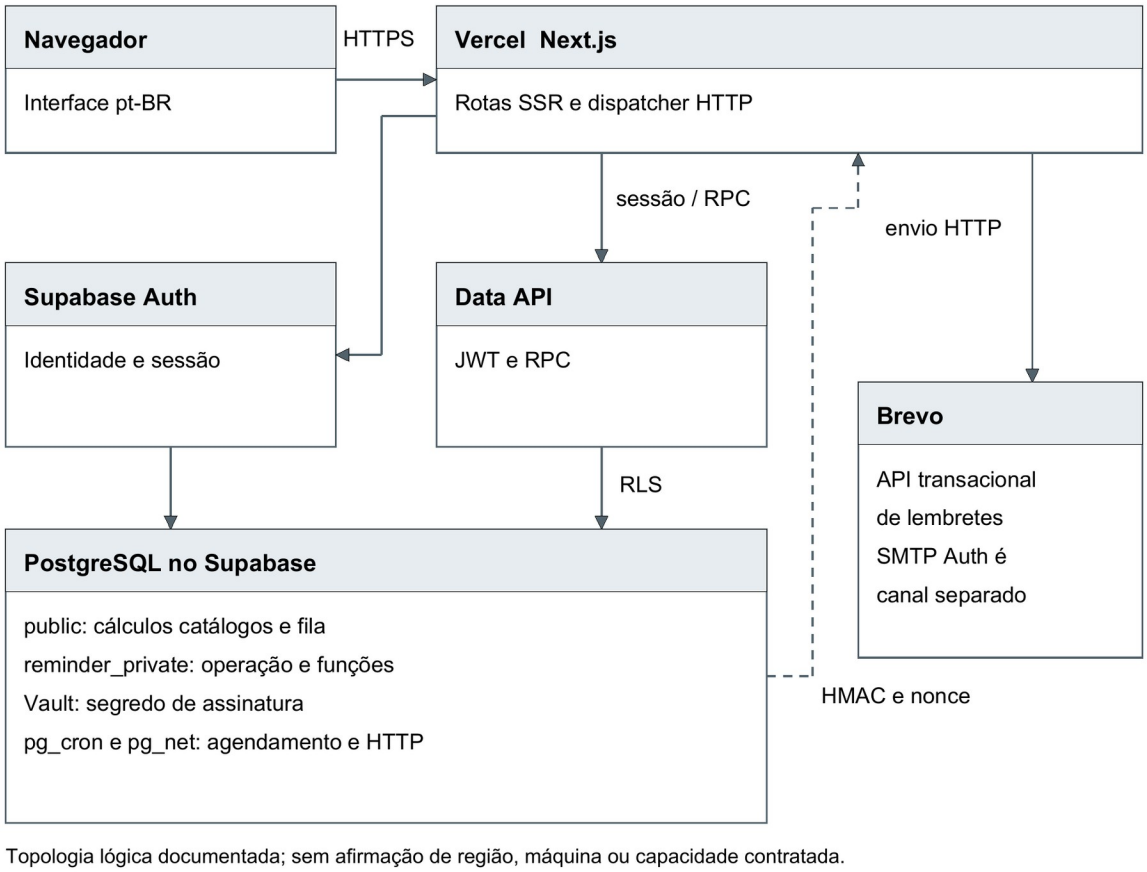
9.4 Aceitação e recebimento

O valor accepted representa confirmação de aceitação pelo serviço de e-mail, não um recibo de leitura ou entrega na caixa. O responsável confirmou em 24/10/2026 que recebeu a mensagem e abriu a memória original na própria conta. Trata-se de aceite operacional individual de um caso hospedado. O relatório incorpora esse resultado sem transformá-lo em medição de latência, disponibilidade ou taxa geral de entrega. [P08]

10 Infraestrutura e operação hospedada

10.1 Componentes e canais

Figura 6 Infraestrutura da aplicação e percurso do envio agendado



Fonte Elaboração a partir da DDL e da implementação versionadas do Prazo Claro.

Vercel hospeda a aplicação Next.js e seus Route Handlers. Supabase fornece Auth, Data API e PostgreSQL. O navegador usa a interface, enquanto o servidor verifica a sessão e encaminha operações sob identidade do usuário. O worker HTTP de lembretes utiliza um cliente privilegiado separado e sem persistência de sessão. Brevo fornece a API transacional para avisos. [P04] [P05]

O SMTP configurado no Supabase atende mensagens de autenticação, como confirmação e recuperação de senha. O aviso de prazo usa a API do Brevo chamada pela aplicação. São canais de configuração e execução distintos, mesmo quando compartilham o provedor. A estrutura não utiliza Edge Function para o dispatcher nem Vercel Cron para esse job: o agendamento está no PostgreSQL, com pg_cron e pg_net. [P03]

Não há conexão SQL direta da aplicação principal por ORM, driver PostgreSQL próprio ou pooler configurado pelo projeto nessa fatia. A Data API é a fronteira de acesso corrente. A

opção local db.pooler.enabled=false não descreve, por si só, a existência ou configuração do serviço de pooling gerenciado de um projeto hospedado.

10.2 Identidade de máquina e proteção do agendamento

pg_cron executa o tick a cada minuto. O tick coleta resultados pendentes, confere habilitação e existência de trabalho Brevo elegível e evita outra invocação da mesma operação ainda em curso nos três minutos anteriores. Somente então cria nonce e timestamp e assina uma mensagem canônica com HMAC SHA-256, usando segredo dedicado guardado no Vault. [P01]

pg_net enfileira uma chamada HTTPS ao endpoint interno com operação e cabeçalhos de timestamp, nonce e assinatura. A chave permanente de assinatura não segue nesses cabeçalhos. O endpoint valida formato, assinatura e idade, aceita defasagem futura de até 30 segundos e idade máxima de 120 segundos e consome o nonce uma única vez pela RPC de serviço antes de processar. O banco confirma operação, timestamp e janela na atualização de used_at. [P01] [P04]

A documentação oficial informa que pg_net inicia a requisição após o commit. Sucesso de uma função que retorna request_id, portanto, não comprova sucesso HTTP ou envio de e-mail. A escolha da assinatura curta foi registrada no ADR 0006 porque a fila de requisições não deve transportar uma credencial duradoura como se fosse um cofre. Administradores do banco e do Vault permanecem privilegiados. [T05] [P03]

10.3 Preparação ativação e suspensão

prepare_scheduler cria configuração desligada e um único job inativo, sem duplicá-lo em chamadas repetidas. O probe pode verificar configuração e provedor sem reservar lembretes nem enviar mensagem. A ativação exige segredo disponível e probe consumido, recente, HTTP 200, com provedor pronto e entrega habilitada, observado nos 15 minutos anteriores. [P01]

set_scheduler_enabled altera a configuração e o estado ativo do job. Suspender o job e a habilitação de envio impede novas execuções elegíveis, mas uma requisição em curso pode terminar. Religar após falha exige diagnóstico da fila e novo probe; não se deve fabricar um estado de sucesso ou antecipar send_at para produzir uma demonstração. O procedimento completo está no runbook versionado. [P05]

10.4 Coleta retenção e observabilidade

collect_scheduler_results correlaciona request_id com a resposta HTTP e guarda resumo permitido. Probes retêm flags de prontidão e códigos fixos. Dispatch retém contagens de

claimed, accepted, failed, uncertain e skipped, com limites validados. A tabela privada não duplica corpo integral da mensagem, e-mail do destinatário, segredo ou assinatura. [P01]

Invocações e execuções do job com mais de sete dias são removidas durante a coleta. Respostas sem conclusão após três minutos recebem marca de ausência. Essa retenção depende de o tick ou uma coleta administrativa executar; não é uma expiração automática independente. A retenção própria do pg_net e a retenção dos históricos dos usuários têm regras e responsabilidades diferentes.

Existem registros de publicação e ensaio, mas este relatório não declara monitoramento contínuo, alertas distribuídos, RPO, RTO, restauração de backup ou recursos computacionais contratados como validados. Região, tamanho de instância, versão remota efetiva, volume de dados e ocupação de disco não foram consultados em 04/11. Esses itens exigem inspeção operacional própria, conforme a finalidade de uso.

10.5 Ambientes e configuração local

Separação de ambientes e evidências

Ambiente	Finalidade	Característica documentada
Desenvolvimento local	Implementação e integração	Supabase em Docker e PostgreSQL major 17 configurado
Banco de CI	Reprodução descartável	Reset local do runner e testes SQL em transação
Preview Vercel	Conferência de build	Configuração isolada, sem herdar credenciais de produção
Produção	Demonstração hospedada	Next.js na Vercel, Supabase remoto e Brevo autorizado

Fonte: config.toml, workflow e evidências de publicação. A tabela não representa consulta atual ao estado de cada serviço. [P05] [P07] [P08]

As portas locais definidas são 54321 para API, 54322 para PostgreSQL, 54320 para banco de sombra de diff e 54323 para Studio. O projeto usa Mailpit nos ensaios locais de e-mail. Realtime está habilitado na configuração, mas não há funcionalidade de negócio que dependa dele nesta entrega. Storage, preferências globais de notificação e sincronização offline não são apresentados como componentes implementados.

Credenciais privilegiadas e segredos de máquina ficam no servidor e no Vault, conforme o caso. O relatório cita nomes e responsabilidades de configuração, sem copiar valores. A chave pública da integração de sessão e a credencial privilegiada de envio não são intercambiáveis.

A rotina local de lembretes exige loopback; o modo Brevo exige o alvo production e o endereço canônico configurados no código.

11 Evolução do schema e entrega

11.1 Sequência das migrations

Versão	Contribuição
20261020001602	Histórico privado e catálogos
20261023172741	Fila e RPCs de lembretes
20261023184137	Modos local e Brevo
20261023192044	Agendador e invocações privadas

Fonte: arquivos SQL e inventário desta edição. Os hashes integrais estão em schema-inventario.json e no manifesto do pacote. [P01]

A primeira migration define quatro tabelas, propriedade, políticas e índices. A segunda introduz a fila e o schema privado com RPCs. A terceira acrescenta delivery_mode e funções por modo, mantendo dados anteriores como local. A quarta acrescenta duas tabelas privadas, instalação condicional de pg_cron e pg_net e as funções do agendador.

As migrations já aplicadas são documentos de evolução e não devem ser editadas como se descrevessem apenas o estado final desejado. Mudanças futuras precisam de novas migrations e verificação de compatibilidade dos dados existentes. Os registros de implantação descrevem a aplicação hospedada e o alinhamento autorizado das versões históricas, sem reset de produção. [P08]

11.2 Reprodução e dependências de versão

O package.json fixa Supabase CLI 2.121.0 e a CI instala dependências com lockfile. Os comandos de banco documentados nesta base pertencem a essa versão. O changelog consultado em 04/11 informa mudanças na CLI v3 para alvo de comandos e confirmação de alterações hospedadas. Não houve atualização da CLI ou de config.toml nesta tarefa; quem reproduzir deve usar a versão fixada e verificar --help antes de adotar uma versão diferente. [P05] [T06]

O workflow do banco inicia serviços locais, aplica reset no runner descartável, executa pgTAP, realiza lint e encerra os serviços. Reset é mecanismo de reprodução desse ambiente e não procedimento de atualização do projeto com dados reais. Os testes inserem fixtures fictícias em transação e fazem rollback, preservando a instalação em uso. [P07]

11.3 Evoluções recomendadas pelo modelo

O crescimento do sistema pode justificar pertencimento a organizações, trilha administrativa de revisão, versões normativas publicadas pelo banco, campos consultáveis derivados e índices de expressão. Essas evoluções precisam preservar a leitura das memórias antigas e demonstrar isolamento equivalente. A migração para ORM, se necessária, deverá explicitar a identidade de cada operação e sua relação com RLS.

Antes de ampliar uso institucional, permanecem decisões de retenção de contas e cálculos, exclusão administrativa, restauração, monitoramento e tratamento de abuso. Para desempenho, medir consultas por proprietário e regime e o custo de paginação antes de acrescentar GIN, particionamento ou novas relações. Não foram criados índices adicionais apenas para constar em um diagrama.

12 Evidências e limites da verificação

12.1 Evidências históricas identificadas

Evidências existentes e alcance

Evidência	Resultado registrado	Alcance
pgTAP de 23/10	146 testes em cinco arquivos	Permissões, RLS, fila, modos e agendador
CI 40061653068	146 SQL, 256 unitários e 67 E2E aprovados; um E2E omitido	Execução identificada da base funcional 9632fd2
Integrações locais reais	Conta, histórico, isolamento e Mailpit registrados	Não equivalem a cada cenário de navegador usar serviço real
Aceite de 24/10	Mensagem recebida e memória aberta na conta	Confirmação humana individual do fluxo hospedado
Inventário de 04/11	DDL lida e artefatos conferidos	Verificação documental, sem reexecutar PostgreSQL

Fonte: registros de testes e aceite do projeto. [P07] [P08]

Os testes SQL cobrem restrição de leitura entre proprietários, impossibilidade de edição de snapshots pela sessão, restrição de catálogos, escrita controlada por RPC, unicidade ativa, concorrência e token de reserva, modos separados, expiração e consumo de nonce. A suíte do agendador usa respostas sintéticas dentro da transação; como não há commit, essas chamadas não comprovam entrega pela Vercel ou pelo Brevo.

A confirmação humana posterior complementa o que esses testes não medem, mas não cria uma pesquisa com participantes nem uma estatística de confiabilidade. Os 146 testes são uma

execução anterior identificada. Esta edição não reaplicou migrations, não executou reset, não consultou dados pessoais e não provocou envio de e-mail.

12.2 Limitações relevantes para a defesa

O principal limite do banco é o contexto acadêmico dos dados preservados. Calendários parciais e revisão jurídica pendente acompanham os snapshots. Ausência de edição por usuário não significa autenticidade criptográfica. Catálogos persistentes ainda não são o canal ativo de administração normativa. A fila evita duplicação silenciosa diante de dúvida, mas não oferece atomicidade entre banco e rede.

Também não há evidência de carga representativa, restauração institucional ou auditoria independente. Um diagrama de implantação não comprova região, recursos de máquina, alta disponibilidade ou capacidade contratada. O relatório distingue essas necessidades das estruturas efetivamente descritas nas migrations e dos resultados já documentados.

13 Conclusão

O banco do Prazo Claro evoluiu de uma concepção ampla para uma implementação individual com persistência de memórias, autorização no PostgreSQL e fila de lembretes. As decisões de engenharia procuram conservar o contexto do cálculo e limitar as escritas privilegiadas a funções com responsabilidade definida. O modelo combina relações para identidade e integridade com JSONB para a memória versionada.

Na infraestrutura, o PostgreSQL participa tanto do armazenamento quanto do agendamento, enquanto o envio HTTP e a verificação de sessão permanecem na aplicação Next.js. A separação entre programação, reserva, aceitação e recebimento permite explicar os limites da operação distribuída. As migrations, os testes e os registros de aceite fornecem uma base rastreável para apresentação e continuidade do TG II.

O recebimento e a abertura da memória do ensaio de 24/10 estão confirmados. As próximas decisões sobre banco dizem respeito a revisão normativa, expansão controlada do domínio e operação institucional, com medição e responsáveis próprios. O estado descrito não pressupõe recursos futuros já instalados.

Apêndice A Consultas de inspeção sem dados pessoais

As consultas abaixo são exemplos de conferência administrativa de metadados e agregados. Foram lidas para este relatório, mas não executadas contra o ambiente hospedado nesta edição. O SQL Editor administrativo não representa a identidade de um usuário comum; testes de RLS precisam estabelecer o papel e os claims apropriados em ambiente isolado.

```
-- Tabelas próprias e ativação de RLS
select n.nspname as schema_name, c.relname as table_name,
       c.relrowsecurity as rls_enabled
from pg_class c
join pg_namespace n on n.oid = c.relnamespace
where c.relkind = 'r'
  and n.nspname in ('public', 'reminder_private')
order by 1, 2;


-- Índices declarados, sem retornar conteúdo de cálculos
select schemaname, tablename, indexname, indexdef
from pg_indexes
where schemaname in ('public', 'reminder_private')
order by schemaname, tablename, indexname;


-- Agregado de operação; acesso administrativo
select delivery_mode, status, count(*) as quantity
from public.deadline_reminders
group by delivery_mode, status
order by delivery_mode, status;


-- Configuração do job, sem listar comandos ou segredos
select jobname, schedule, active
from cron.job
where jobname = 'prazo-claro-reminders';
```

O diagnóstico deve limitar resultados, não exportar cabeçalhos da fila HTTP ou conteúdo do Vault e não copiar e-mails, tokens ou snapshots privados para evidências públicas. Esses cuidados fazem parte da minimização já adotada nos registros operacionais do projeto.

Apêndice B Perguntas para explicar o modelo

Por que usar JSONB em vez de uma tabela para cada dia

A memória é lida e exportada como um conjunto com versões e avisos. JSONB conserva esse agregado sem exigir joins por evento ou dia. Tabelas de detalhe podem ser introduzidas se consultas analíticas justificarem a mudança, mantendo o snapshot histórico e uma migração compatível.

Por que o banco não calcula o prazo

O motor foi separado para produzir o mesmo resultado com as mesmas entradas e versões, sem rede ou relógio. O banco verifica integridade e acesso, preserva o resultado e programa avisos. A lógica processual não foi duplicada em funções SQL.

O banco suporta equipes e cadastro completo de processos

O modelo conceitual de destino prevê essa expansão, mas as tabelas de organizações, memberships e processos não existem no schema entregue. A propriedade corrente é individual. Uma implementação futura precisará de novas políticas e testes entre organizações.

O que comprova que um lembrete chegou

accepted comprova aceitação pelo provedor. Neste projeto, o recebimento e a abertura da memória de um ensaio foram confirmados pelo responsável em 24/10. A confirmação não é inferida somente de uma linha da fila ou de um job concluído.

Apêndice C Catálogo de funções

O inventário final contém dez funções públicas de fronteira e 12 funções internas. O quadro apresenta segurança de execução e responsabilidade. As concessões pertencem às migrations: usuário designa authenticated, serviço designa service_role e administração designa o proprietário administrativo, sem autorização de chamada para o cliente comum. Todas as 22 definições finais fixam search_path vazio. [P01]

Função	Segurança	Papel e responsabilidade
public.cancel_deadline_reminders	invoker	Usuário. Cancela um ou todos os ativos próprios.
public.claim_deadline_reminders	invoker	Serviço. Reserva lote no modo local.
public.list_deadline_reminders	invoker	Usuário. Geração mais recente por cálculo sob proprietário.
public.prepare_deadline_reminder	invoker	Serviço. Prepara destinatário e memória após validar token.
public.schedule_deadline_reminder	invoker	Usuário. Agenda uma geração local.
public.settle_deadline_reminder	invoker	Serviço. Registra resultado do processamento.
reminder_private.cancel	definer	Usuário via wrapper. Confere identidade e cancela ativos.

Função	Segurança	Papel e responsabilidade
reminder_private.claim	invoker	Serviço via wrapper. Delega claim_for_mode para local.
reminder_private.prepare	definer	Serviço via wrapper. Token lease conta e snapshot.
reminder_private.schedule	definer	Usuário via wrapper. Identidade confirmada e programação futura.
reminder_private.settle	definer	Serviço via wrapper. Liquidação condicionada ao token.
public.claim_deadline_reminders_for_mode	invoker	Serviço. Reserva lote no modo solicitado pelo servidor.
public.list_deadline_reminders_for_mode	invoker	Usuário. Listagem própria restrita ao modo.
public.schedule_deadline_reminder_for_mode	invoker	Usuário. Programa geração em modo explícito.
reminder_private.claim_for_mode	definer	Serviço via wrapper. Expiração e reserva transacional por modo.
reminder_private.schedule_for_mode	definer	Usuário via wrapper. Agenda e fixa modo na transação.
public.consume_reminder_invocation	invoker	Serviço. Consome nonce e operação uma única vez.
reminder_private.collect_scheduler_results	definer	Administração ou tick interno. Coleta HTTP e aplica retenção.
reminder_private.consume_invocation	definer	Serviço via wrapper. Consumo atômico de nonce válido.
reminder_private.dispatch_tick	definer	Administração ou Cron. Assina e enfileira probe ou dispatch.
reminder_private.prepare_scheduler	invoker	Administração. Configuração desligada e job único inativo.
reminder_private.set_scheduler_enabled	invoker	Administração. Habilita ou suspende sob condições de probe.

Referências e rastreabilidade

P01 Migrations e inventário físico

supabase/migrations/20261020001602_private_history_and_versioned_catalogs.sql
supabase/migrations/20261023172741_deadline_reminders.sql
supabase/migrations/20261023184137_reminder_delivery_modes.sql
supabase/migrations/20261023192044_reminder_scheduler.sql
docs/relatorio-banco/schema-inventario.json

P02 Domínio requisitos e arquitetura

GLOSSARY.md
docs/requirements.md
docs/architecture.md
supabase/seed.sql

P03 Decisões arquiteturais

docs/architecture/adr/0002-supabase-data-api-sem-orm-inicial.md
docs/architecture/adr/0003-historico-individual-antes-de-organizacoes.md
docs/architecture/adr/0005-fila-de-lembretes-e-envio-incerto.md
docs/architecture/adr/0006-agendamento-com-assinatura-descartavel.md

P04 Implementação de aplicação

src/app/api/history/route.ts
src/application/history-snapshot.ts
src/application/history-query.ts
src/app/api/reminders/route.ts
src/app/api/internal/reminders/dispatch/route.ts
src/lib/supabase/server.ts
src/lib/supabase/http.ts
src/lib/reminders/config.ts

P05 Configuração entrega e operação

supabase/config.toml
package.json
.github/workflows/ci.yml
docs/reminder-scheduler-operations.md
docs/email-reminders-external.md

P06 Segurança e preservação de dados

docs/security/README.md
docs/security/release-review.md

P07 Testes e evidências SQL

supabase/tests/catalogs_rls.test_test.sql
supabase/tests/deadline_calculations_rls.test_test.sql
supabase/tests/reminder_modes.test_test.sql
supabase/tests/reminder_scheduler.test_test.sql
supabase/tests/reminders.test_test.sql
docs/testing/reminder-scheduler.md
docs/testing/reminder-delivery-modes.md
docs/testing/signup-confirmation-page.md
P08 Publicação e aceite humano

docs/testing/reminders-production.md
docs/testing/reminder-receipt-2026-10-24.md
docs/entrega/aceite.md
T01 POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17 Documentation. JSON Types. Consulta em 04/11/2026.

https://www.postgresql.org/docs/17/datatype-json.html
T02 SUPABASE. Row Level Security. Consulta em 04/11/2026.

https://supabase.com/docs/guides/database/postgres/row-level-security
T03 POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17 Documentation. Constraints e Partial Indexes. Consulta em 04/11/2026.

https://www.postgresql.org/docs/17/ddl-constraints.html
https://www.postgresql.org/docs/17/indexes-partial.html
T04 POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17 Documentation. SELECT e cláusula SKIP LOCKED. Consulta em 04/11/2026.

https://www.postgresql.org/docs/17/sql-select.html
T05 SUPABASE. pg_net Async Networking e Cron. Consulta em 04/11/2026, incluindo pesquisa no plugin oficial.

https://supabase.com/docs/guides/database/extensions/pg_net
https://supabase.com/docs/guides/cron
T06 SUPABASE. Changelog. Anúncio de 04/11/2026 sobre CLI v3 e mudanças de alvo e confirmação. Consulta em 04/11/2026.

https://supabase.com/changelog

Diagramas gerais ampliados no PDF único

As duas páginas finais do PDF apresentam pranchas em formato A3 horizontal. A primeira reúne as sete tabelas próprias e suas 65 colunas, além da referência à identidade gerenciada auth.users, com as chaves e os quatro relacionamentos físicos. A segunda apresenta a arquitetura completa da entrega: interface, APIs, domínio, catálogos, autenticação, persistência, agendamento, envio de mensagens, serviços externos, desenvolvimento e entrega. Esses diagramas complementam as figuras de cada capítulo e podem ser ampliados no leitor de PDF sem perda de nitidez. Não representam um inventário de servidores ou capacidades contratadas.

Diagrama geral completo do banco de dados

Todas as tabelas e colunas próprias, chaves e quatro relacionamentos impostos por FK.

FK owner_id

public.legal_sources

PK id : uuid
title : text
url : text
provision : text
consulted_at : timestamptz
review_status : text
created_at : timestamptz

1 : 0..N

public.legal_rule_versions

PK id : uuid
rule_key : text
version : text
FK source_id : uuid
scope : jsonb
valid_from : date
valid_until : date [nulo]
definition : jsonb
review_status : text
created_at : timestamptz

UK (rule_key, version)

public.calendar_versions

PK id : uuid
calendar_key : text
version : text
court : text
locality : text
coverage_start : date
coverage_end : date
is_complete : boolean
review_status : text
snapshot : jsonb
created_at : timestamptz

UK (calendar_key, version)

auth.users [gerenciada pelo Supabase]

PK id : uuid
Identidade, e-mail e estado da conta.
Demais campos pertencem ao Supabase Auth.

1 : 0..N

public.deadline_calculations

PK id : uuid
FK owner_id : uuid
title : text
input : jsonb
result_snapshot : jsonb
status : text
created_at : timestamptz

1 : 0..N

public.deadline_reminders

PK id : uuid
FK calculation_id : uuid
FK owner_id : uuid
due_date : date
lead_days : integer
send_hour : integer
time_zone : text
send_at : timestamp with time zone
snapshot_metadata : jsonb
status : text
attempt_count : integer
next_attempt_at : timestamp with time zone
claim_token : uuid [nulo]
claimed_at : timestamp with time zone [nulo]
accepted_at : timestamp with time zone [nulo]
error_code : text [nulo]
created_at : timestamp with time zone
delivery_mode : text

UK parcial: uma geração ativa por cálculo

reminder_private.scheduler_config

PK singleton : boolean
enabled : boolean

CHECK singleton = true; inicia desligado

reminder_private.scheduler_invocations

PK nonce : uuid
issued_at : bigint
operation : text
created_at : timestamp with time zone
used_at : timestamp with time zone [nulo]
request_id : bigint [nulo]
http_status : integer [nulo]
completed_at : timestamp with time zone [nulo]
timed_out : boolean [nulo]
summary : jsonb

UK request_id; sem FK para pg_net

Leitura do modelo e fronteiras

7 tabelas próprias; 65 colunas.
4 FKs: setas do pai para o filho.
Exclusão do usuário: cascata nos próprios cálculos e lembretes. Fonte: RESTRICT.
PK = chave primária; UK = unicidade.
[nulo] = coluna que aceita NULL.
Sem FK entre JSONB e catálogos.
Sem FK entre scheduler e fila HTTP.
Auth / cron / net / Vault são gerenciados.
Dicionário: cap. 5; restrições e índices: cap. 6.
RLS habilitada nas sete tabelas próprias.

Diagrama completo da arquitetura do sistema

Aplicação, domínio, persistência, autenticação, lembretes, serviços externos e entrega.

